II. SQL (Structured Query Language)

SQL ist die Datenbanksprache. Mit dieser können Datenbanken erstellt und verändert werden. Außerdem können Abfragen vorgenommen werden. Jeder Zugriff aus der Anwender-Software wird in einen SQL-Befehl umgewandelt – auch jedes Like, das du in einer App setzt.

1. Datenbankabfragen mit SQL

In einer relationalen Datenbank werden Informationen in Tabellen (**Klassen**) repräsentiert. Die Zeilen dieser Tabelle heißen **Objekte**, die Spalten **Attribute**. Am Anfang hat dein Netzwerk nur eine einzige Klasse: **users**.

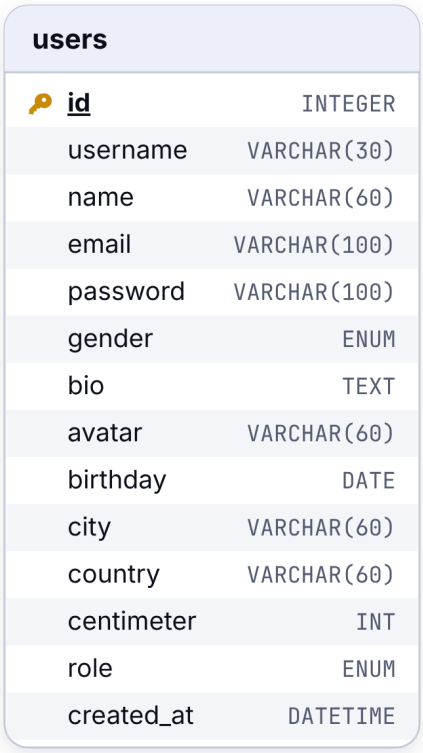


Abb. 4: Die Klasse users im Diagramm von YourSozialNet

So sehen einige Objekte der Klasse users aus:

Klasse (Tabelle): users

id	username	name	gender	birthday	city	centimeter
2	noobqueen	Mia Sander	female	2009-09-16	Kiel	165
4	levi.kicks	Levi Schumacher	male	2009-04-11	**Kiel**	184
6	coach_ralf	Ralf Opitz	male	1980-07-10	Kiel	179
8	alina.daily	Alina Keller	female	2005-08-22	Hamburg	170
14	inkandmila	Mila Hoffmann	female	2010-04-21	Leipzig	160
37	levi.berlin	Levi Schumacher	male	1992-05-03	Berlin	181

← **Objekt** = eine Zeile, ein Datensatz

🔑 **Primärschlüssel** id: jede Zeile hat eine andere **Attribut** = eine Spalte, z. B. city

Abb. 5: Ausschnitt aus der Tabelle users

Damit in der Datenbank nicht händisch nach Informationen gesucht werden muss, können mit SQL-Befehlen die Inhalte der Datenbank gezielt ausgewählt oder sortiert werden. Das Ergebnis einer SQL-Abfrage ist immer wieder eine Tabelle, welche sich aus der Abfrage ergibt.



Abb. 6: Eine SQL-Abfrage wählt Spalten und Zeilen aus

Dabei folgen SQL-Befehle immer einer bestimmten Grundstruktur:

- Der **SELECT**-Befehl gibt an, welche Attribute ausgegeben werden sollen. Mit einem ***** werden alle Attribute einer Klasse ausgegeben.
- Im **FROM**-Befehl wird die Klasse angegeben, welche betrachtet wird.
- Im **WHERE**-Befehl können noch weitere Bedingungen gesetzt werden.

AUFBAU

```
SELECT <Liste von Attributen>
FROM <Klasse>
WHERE <Bedingung>
```

BEISPIEL · PERSONEN, DIE KLEINER ALS 1,60 M SIND

```
SELECT id, name, centimeter
FROM users
WHERE centimeter < 160
```

1.1 Projektion (Ausdrücke und Auswahl von Attributen)

Bei einer Projektion wird eine Klasse gezielt auf bestimmte Spalten reduziert. Es können auch Spalten neu gebildet werden, wenn Inhalte zusammengefasst oder umgerechnet werden sollen. Diese zusätzlichen Spalten werden mit dem Keyword **AS** benannt. Ausdrücke können Attribute, konstante Werte, arithmetische Operationen (+, -, *, /) oder Funktionen (`NOW()`) sein.

AUFBAU

```
SELECT <Liste von Attributen>,
       <Ausdruck> AS <Name>
FROM <Klasse>
```

BEISPIEL · GRÖSSE IN METER UMGERECHNET

```
SELECT id, name,
       centimeter / 100 AS meter
FROM users
```

Funktion	Bedeutung
`NOW()`	liefert das aktuelle Datum mit Uhrzeit
`DATEDIFF(<datum1>, <datum2>)`	liefert die Anzahl der Tage, die zwischen den beiden Daten liegen
`FLOOR(<x>)`	rundet `<x>` ab auf eine ganze Zahl
`YEAR(<datum>)`	liefert das Jahr eines Datums (genauso `MONTH`, `DAY`, `HOUR`)

Aufgabe 1

Notiere für die folgenden Abfragen einen SQL-Befehl. Deine Ausgabe sollte eine geeignete Auswahl an Attributen haben. Halte diesen digital oder analog fest!

a) Gib alle Einträge aus der Tabelle users aus.

b) Gib alle Benutzernamen (username) und echten Namen (name) aller Einträge aus users aus.

c) Gib die Wohnorte aller Mitglieder aus.

d) Gib die Größe der User in Fuß (feet) an (1 cm = 0,0328 ft).

e) Berechne das Alter der User und zeige den Namen und das Alter an. Nutze die Funktionen von der vorherigen Seite.

f) Überprüfe mit deiner Lösung das Alter von Noah (Username noah.sz). Stimmt dein Ergebnis? Falls nicht, überlege dir, woran das liegen könnte.

1.2 Sortierung und Doppelungen

DISTINCT kann nach dem **SELECT**-Befehl eingefügt werden. Dieser verhindert Doppelungen in der Ausgabe. Es werden identische Zeilen zusammengefügt. Dies ist praktisch, wenn nur bestimmte Attribute betrachtet werden. Beispielsweise können so alle Länder ausgegeben werden.

AUFBAU

```
SELECT DISTINCT <Liste von Attributen>
FROM <Klasse>
```

BEISPIEL · HERKUNFTSLÄNDER DER USER

```
SELECT DISTINCT country
FROM users
```

ORDER BY – Um Ergebnisse nach einem Attribut zu sortieren, kann der Befehl **ORDER BY <Attribut>** angefügt werden. Dieser sortiert die Liste alphabetisch bzw. nach Größe. **ASC** sorgt für eine ansteigende und **DESC** für eine absteigende Sortierung.

LIMIT – Mit **LIMIT <Anzahl>** wird die Ausgabe auf die ersten Zeilen begrenzt. Zusammen mit ORDER BY erhältst du so z. B. die „Top 3“.

AUFBAU

```
SELECT <Liste von Attributen>
FROM <Klasse>
ORDER BY <Attribut> ASC/DESC
LIMIT <Anzahl>
```

BEISPIEL · DIE DREI GRÖSSTEN USER

```
SELECT name, centimeter
FROM users
ORDER BY centimeter DESC
LIMIT 3
```

Aufgabe 2

Notiere für die folgenden Abfragen einen SQL-Befehl.

a) Gib alle Wohnorte in einer sortierten Liste aus, bei der jede Stadt einmalig auftritt.

b) Ordne die Mitglieder nach ihrer Größe absteigend.

c) Kehre die Reihenfolge der Tabelle **users** um.

d) Gib die fünf zuletzt registrierten User aus.